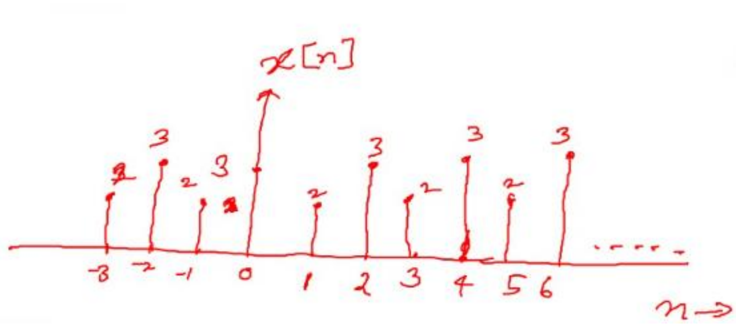
	Marwadi University **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim:** Generate Signals	
Openended-Problem: 01	**Date:09-07-2026**	**Enrollment No:92400133121**

1.

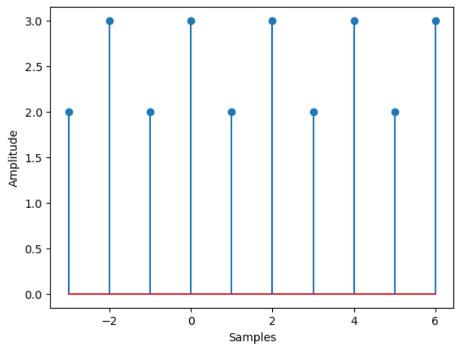


Code:

```
import numpy as np
import matplotlib.pyplot as plt

def generate_signal(n):
    signal = []

    for i in range(len(n)):
        if i % 2 == 0:
            signal.append(2)
        else:
            signal.append(3)

    return signal

n = np.arange(-3, 7, 1)
x = generate_signal(n)

plt.stem(n, x)
plt.xlabel('Samples')
plt.ylabel('Amplitude')

plt.show()
```

Output:

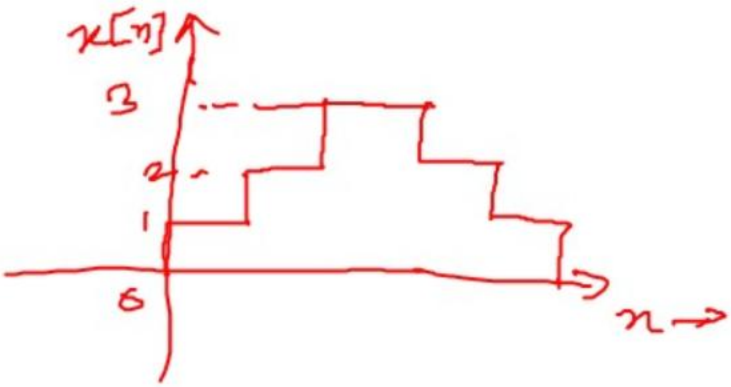
	Marwadi University **Faculty of Engineering & Technology** **Department of Information and Communication Technology**	
Subject: DSIP (01CT1513)	**Aim:** Generate Signals	
Openended-Problem: 01	**Date:09-07-2026**	**Enrollment No:92400133121**



Conclusion:

The amplitude of this particular signal is 2 when the value of n is odd and 3 in case of even values.

2.

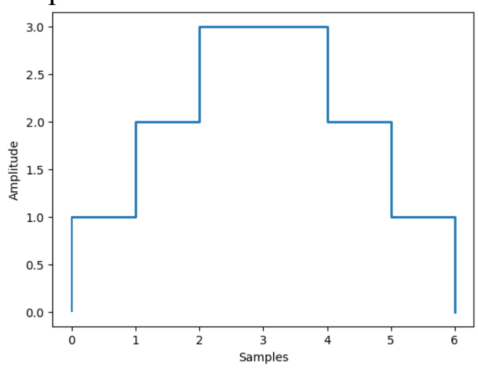


Code:

```
import numpy as np
import matplotlib.pyplot as plt

def generate_signal():
    signal = [1, 2, 3, 3, 2, 1, 0]
    return signal

n = np.arange(0, 7, 1)

x = generate_signal()

plt.step(n, x, where='post', linewidth=2)

plt.vlines(0, 0, 1)
```

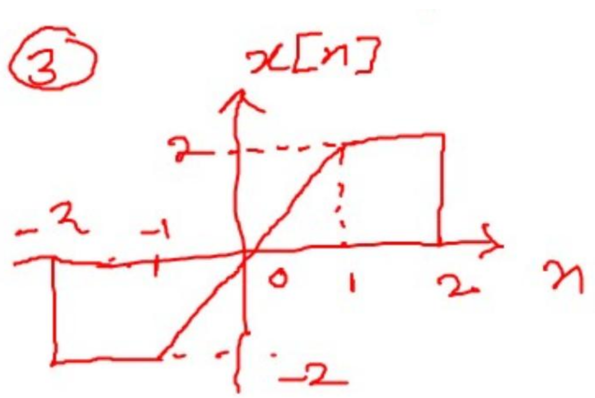
```
plt.xlabel('Samples')
plt.ylabel('Amplitude')

plt.show()
```

Output:



Conclusion:
It is a stair case signal where the amplitude is increasing till amplitude 3, remaining constant and then decreasing.

3.

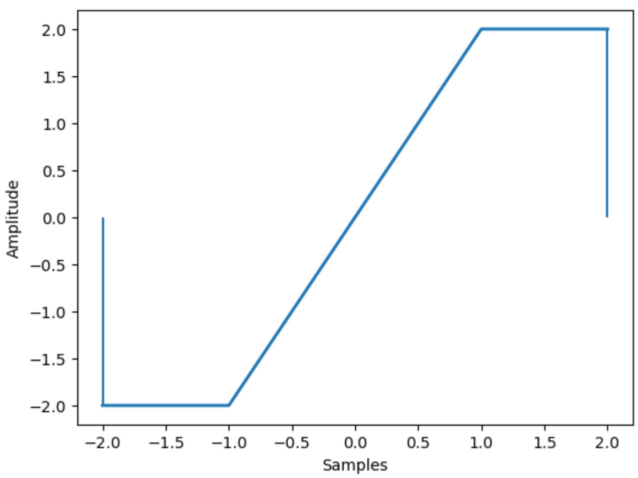


Code:

```
import numpy as np
import matplotlib.pyplot as plt

def generate_signal():
    signal = [-2, -2, 0, 2, 2]
    return signal

n = np.arange(-2, 3, 1)
```

```
x = generate_signal()

plt.plot(n, x, linewidth=2)

plt.vlines(-2, 0, -2)
plt.vlines(2, 2, 0)

plt.xlabel('Samples')
plt.ylabel('Amplitude')

plt.show()
```

Output:



Conclusion:
In this signal the amplitude is constant at -2 at n = -2, increases from -1 to 1 till +2 ,remains constant and then drop to 0

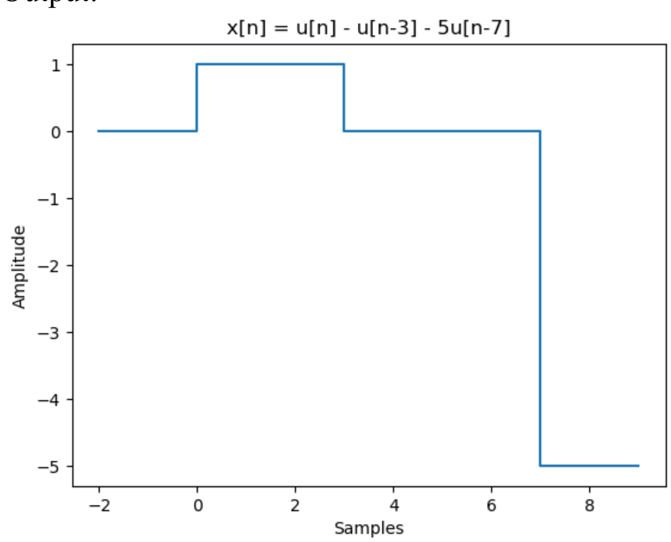
4. $x[n]=u[n]-u[n-3] - 5u[n-7]$

Code:

```
import numpy as np
import matplotlib.pyplot as plt

def simulate_function(n):
    x = np.zeros_like(n)

    x[n >= 0] = 1
    x[n >= 3] -= 1
```

```
    x[n >= 7] -= 5

    return x

n = np.arange(-2, 10, 1)

signal = simulate_function(n)

plt.step(n, signal, where='post')
plt.title('x[n] = u[n] - u[n-3] - 5u[n-7]')
plt.xlabel('Samples')
plt.ylabel('Amplitude')

plt.show()
```

Output:



Conclusion:
The amplitude of signal is 0 for n<0 , 1 for 0<=n<3 , 0 for 3<=n<7 then -5 at n=7 onwards

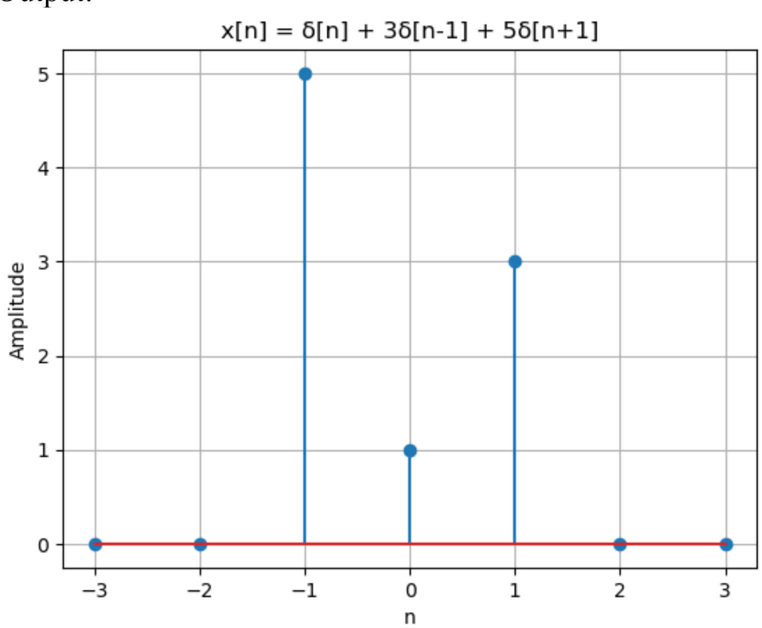
5. $x[n] = \delta[n] + 3\delta[n-1] + 5\delta[n+1]$

Code:

```
import numpy as np
import matplotlib.pyplot as plt

def simulate_function(n):
    x = np.zeros_like(n)
```

```
    x[n == 0] = 1
    x[n == 1] = 3
    x[n == -1] = 5

    return x

n = np.arange(-3, 4, 1)
signal = simulate_function(n)

plt.stem(n, signal)
plt.title('x[n] = δ[n] + 3δ[n-1] + 5δ[n+1]')
plt.xlabel('Samples')
plt.ylabel('Amplitude')

plt.show()
```

Output:



Conclusion:
The signal will have 3 impulses at n = -1,0,1 with their respective amplitudes 5,1,3.At remaining samples the amplitude will be 0